

Small workflow runners and one observing server

Design proposal for Gimbal issue 439 · October 9, 2026

Execution and data flow

Compile project workflows into a small executable. Run each invocation in its own process. Let the central server subscribe to that process over HTTP, with events streamed as they happen and controls sent as ordinary requests. Use Unix sockets locally. The same application protocol can later use a network connection. Tens of runs need neither a broker nor a scheduling service.

The server initiates the connection; the runner pushes events down the open response. This is a subscription, not repeated polling. It gives us one direction for connection establishment, straightforward request/reply controls, and ordinary Go net/http handlers. Runner-initiated uploads would also work, but would make every runner manage a destination and reconnect policy while still requiring a return path for controls. Choose subscriptions while the server can reach the runners.

Browser / CLI → central server → runner: controls and artifact reads
→ runner: open event subscription
← runner: ordered events, continuously

The runner owns the workflow stack, typed `Generate[T]` results, agent sessions, cancellation, cleanup, and recording. The server owns project discovery, launching, the run index, observation reduction, browser snapshots, and browser SSE. Neither browser connections nor IPC request contexts own the workflow lifetime. Cancelling a run is an explicit command.

Keep the web application out of the runner's import graph. A generated project entry point links its workflows, runtime, selected adapters, recorder, and small HTTP handler. It does not import web, skgo, embedded frontend assets, the built-in workflow catalog, or the source analyzer. Ordinary HTTP does not require loading the web application. Build-time analysis emits command descriptions and design data as sidecar files; typed schemas stay with executable workflow code.

The shared CLI constructs commands from that description, preserving flags, help, validation, and role selection; the executable validates inputs again before execution. The server renders project-specific design metadata through its common frontend. Identify a workflow by project plus name, and bind each run to its executable build and a saved copy of its design. Updating project code builds another executable without rebuilding Gimbal. Local compilation is the first delivery option; prebuilt bundles use the same boundary.

Removing web assets alone does not solve memory growth. Today `run.go` creates an observation Store even without a server; it retains completed turns and transcripts. **Move that projection to the central server.** The runner writes a new `events.jsonl`: lifecycle and agent payloads with placement, native references, and one monotonic journal position. Existing `run.jsonl` and session logs retain their per-log Seq meanings and human-readable records. This deliberately duplicates recording on disk to provide one ordered IPC replay source without retaining UI state. All owned recording errors retain the existing run-error contract. Derived observation snapshots and deltas move to the server.

A single writer establishes event order and publishes only complete records. Readers stream from disk with bounded buffers; a slow server must not accumulate an unbounded runner queue or hold up agent execution. Runtime state and active adapter state still consume memory. The claim is narrower: no web application and no retained historical UI projection in each runner.

Discovery, recovery, and controls

The central launcher reserves the run ID and directory and passes them with inputs and bindings. The generated runner calls a revised Run/OpenRun that accepts this assignment: a deliberate API change replacing today's self-assigned identity and unsupported-sharing caveat with exclusive per-run ownership. Runners stop writing project.jsonl; the server maintains the project index from their events and retains owner.lock. CLI launches go through this launcher. Once ready, the runner publishes its endpoint and build identity; the server verifies them live. On startup it scans retained descriptors to reconnect. A descriptor or PID alone proves nothing.

GET events streams newline-delimited JSON journal records after a supplied position, then waits for new records without a replay/live gap. The server persists its prefix under its own instance directory, projects/<project>/runs/<run>/events.jsonl, before advancing its cursor. It reduces that copy into the existing browser snapshot-plus-SSE contract. Reconnect resumes after that cursor; repeated positions are ignored and gaps are exposed. Runner sequence numbers and browser projection cursors are separate. A fresh server can replay from the beginning; snapshots and compaction can wait until replay cost justifies them.

A broken connection marks the run **unreachable**, not completed. Heartbeats distinguish an idle workflow from a dead connection. Terminal events record the workflow result, while cleanup status determines whether controls must remain available. Keep the runner reachable while owned cleanup is pending. After local exit, it imports any missing suffix from the runner's retained journal; server restart replays the server copy. Lost observation connectivity does not cancel execution. Failure to write the runner's authoritative journal retains the public Run contract: recording errors enter the returned run error. Server projection failures are reported separately.

The IPC surface follows controls we already have:

Exchange	Meaning
Identity and status	Verify run/build/protocol identity; report execution and cleanup state.
Event subscription	Lifecycle, transcript events, usage, pending interviews, and control outcomes.
Session steer; turn interrupt	Target current session/turn IDs; preserve landed, dropped, and error distinctions.
Loop steer; interview answer	Queue planner input or answer the exact pending question ID.
Whole-run cancel	Request cancellation and retry retained cleanup; acceptance is not proof of cessation.
Artifact read	Fetch run-owned values and saved design data by run-relative identity.

The IPC handler accepts a command ID, calls the existing controller, then appends a control_result journal envelope with that ID and the returned outcome/error. This leaves public lifecycle payloads and adapter signatures unchanged. After a timeout, match the ID against the journal; absence means unknown. Do not automatically retry steers or answers. Cancellation acceptance still differs from cleanup completion. No durable command queue is needed. Reconcile uncertain launches by their reserved run ID before launching again.

For containers and other machines, keep paths local to the runner and transfer observation data and referenced artifacts through the protocol. A remote process must retain its journal and artifacts on persistent storage until imported; an ephemeral container cannot disappear after its final event. Remote launch, archive transfer, and deletion policy are later work. Reachable HTTPS or a tunnel can carry the same requests; use authenticated transport before exposing it beyond local sockets. Outbound-only networks may eventually need a runner-initiated tunnel. That changes connection establishment, not event or control semantics.

The first implementation should prove project command discovery, launch, live observation, interview/steer/cancel, completed-run replay, and two projects with the same workflow name. Check the runner's dependency graph and measure its idle and transcript-heavy memory separately from agent subprocesses. Demonstrate that a slow or disconnected observer does not stall the workflow or grow its IPC queue. Keep remote deployment, mixed-version compatibility, and seamless upgrades outside that first gate; reject incompatible protocol versions clearly. The substantive change is separating the recorder from the projection and hosted ownership - not introducing a new workflow language or distributed platform.